

## **True Random Number Generator on Raspberry Pi Crypto Engine**

### **Introduction**

Random numbers are really important in modern cryptography because secure systems depend on values that cannot be predicted or repeated. True Random Number Generators, or TRNGs, help with this since they rely on physical processes instead of math formulas, which makes their output naturally unpredictable. Because of that, TRNGs show up in a lot of security applications like encryption, authentication, digital signatures, and key generation. For this midterm project, our group worked with the TRNG built into the Raspberry Pi's crypto engine. We collected raw random values from the device, ran a few simple tests on them, made some plots, and checked whether the hardware was actually producing random data the way it should. We also compared what we observed to what we learned about different types of TRNGs.

### **Types of TRNGs**

There are several main categories of TRNGs, each based on a different physical entropy source. The first major category is noise based TRNGs. These generators rely on naturally occurring electrical noise inside components. Examples include thermal noise in resistors, avalanche noise in reverse biased diodes, and shot noise produced by electron movement. The noise is amplified, sampled using an analog to digital converter, and then post processed. This method is extremely common in microcontrollers and hardware security modules.

A second category is oscillator jitter TRNGs. These rely on timing variations in oscillators, such as ring oscillators or PLLs. The small unpredictable shifts in timing, which occur due to temperature changes, voltage fluctuations, and semiconductor imperfections, create entropy. When one oscillator is sampled by another, the jitter produces bits that cannot be predicted. This type is popular in FPGAs and modern processors because it can be implemented entirely using digital logic.

A third category is metastability based TRNGs. These are built using circuits that can enter unstable or metastable states. When a metastable circuit resolves, the outcome depends on tiny variations in the environment. This unpredictability becomes the entropy source. This method is sometimes used in ASICs and custom hardware security blocks.

A fourth category is quantum TRNGs. These use quantum mechanical properties such as photon behavior or electron spin. Because quantum events are fundamentally unpredictable, this category provides extremely strong randomness. However, these generators require specialized equipment and are not typically found in general purpose devices.

Finally, many modern systems use hybrid TRNGs. These combine multiple sources of entropy and apply health tests and conditioning algorithms to guarantee a reliable output. Hybrid designs help protect against hardware failure and environmental influences.

### **TRNG Used in the Midterm**

The Raspberry Pi has a built-in hardware random number generator inside its crypto co-processor. The exact design isn't fully documented, but it's generally understood to use some type of electronic noise source along with digital conditioning to clean up the output. Linux

makes this TRNG available through the file `dev hwrng`, and reading from it gives a steady stream of 32-bit random values produced by the hardware.

To gather data from the TRNG, we wrote a short C program that opens `dev hwrng`, reads 32-bit numbers in a loop, and prints them out in hexadecimal form. We redirected the output into a file called `trng_output.txt`, which ended up containing thousands of random values. Once we had the data, we used Python scripts to turn the hex values into bits and bytes so we could analyze them. We also generated plots to check how the values were distributed and to see whether the TRNG appeared to be producing uniform, unbiased output.

### **Statistical Evaluation of the TRNG Output**

We ran two main tests to check whether the TRNG was actually working the way it should. The first one was a bit distribution test. We took all the random numbers we collected and broke them down into individual bits, then counted how many zeros and ones there were. The results were almost perfectly balanced, and the plot showed that both bit values showed up about the same amount. That's exactly what you expect from a real TRNG, since any major imbalance could mean something is wrong or biased. In our case, there was no noticeable issue.

The second test was a byte distribution test. Here we looked at every possible byte value from 0 to 255 and counted how often each one appeared. The TRNG covered the whole range, and the frequencies were fairly even with normal randomness variation. There were no gaps, no strange spikes, and no repeating patterns. Seeing a spread like this is a good sign that the TRNG output is uniform. Putting both tests together, the results show that the values coming from `dev hwrng` have strong entropy and don't show any obvious structure or predictability.

### **Relation to TRNG Theory**

Based on what is known about the Raspberry Pi's crypto engine and the results we got from our tests, the hardware RNG we used behaves like a typical noise-based TRNG. The output patterns we saw are consistent with how electronic noise sources usually look once they've gone through some digital conditioning. The balanced number of zeros and ones, the fairly even spread of byte values, and the lack of any obvious patterns all match what you would expect from a solid TRNG.

While our testing wasn't as in-depth as something like the official NIST SP 800-22 test suite, the results we gathered are still strong enough to show that the Raspberry Pi's TRNG is producing real entropy. For the scope of this midterm, the simple checks we ran are enough to confirm that the hardware RNG is functioning the way it should.

### **Conclusion**

For Midterm 2, our group was able to implement and test a True Random Number Generator using the Raspberry Pi's hardware crypto engine. We collected the random values it generated, saved them to a file, and ran a few statistical checks to see how the output behaved. The bit and byte distribution results showed that the TRNG was producing data that looked unbiased and unpredictable, which is what we would expect from a real hardware-based TRNG. Working on this project gave us hands-on experience with hardware entropy sources, Linux device files, reading and processing raw data, and analyzing randomness. From everything we observed, we feel confident saying that the Raspberry Pi's TRNG is working properly and producing high quality random numbers.